National University of Computer and Emerging Sciences

FAST School of Computing **Fall-2022** **Islamabad Campus**

Question 1 [20 Marks]

1. Consider the following (partially completed) code snippet for Breath First Traversal (or Leve Order Traversal) of a tree and answer the following questions. [7 marks]

```
01: void printLevelOrder(Node* root)
02: {
03:     if (______________________________)
04:         return;
05:     queue<Node*> q;
06:     q.enqueue(root);

07:     while (______________________________) {
08:         Node* node = q.front();
09:         cout << node->data << " ";

10:         ______________________________

11:         if (______________________________)

12:             q.enqueue (______________________);

13:         if (______________________________)

14:             q.enqueue (______________________);
15:     }
16: }
17: int main()
18: {
    . . .
19:     printLevelOrder(root);
20:     return 0;
21: }
```

a. Select the correct option for line 03 of the `printLevelOrder` method above. [1 mark]

i. `root = = NULL`

ii. `q.empty( ) == true`

iii. `root != NULL`

iv. `q.empty() == false`

v. None of above

15

b. Select the correct option for line 07 of the `printLevelOrder` method above. [1 mark]

i. `q.front( ) == root`

ii. `q.empty( ) == true`

iii. `q.dequeue( ) == root`

iv. `root != NULL`

v. `q.empty() == false`

vi. None of above

1

c. Select the correct option for line 10 of the `printLevelOrder` method above. [1 mark]

i. `q.front( )`

ii. `q.dequeue ( )`

iii. `root = q.dequeue( )`

iv. `delete(node)`

v. `root = NULL`

vi. Not needed

1

d. Select the correct option for line 11 of the `printLevelOrder` method above. [1 mark]

i. `Node->left = q.front( )`

ii. `node->right != NULL`

iii. `node->left != NULL`

iv. `node != NULL`

v. `root == NULL`

vi. Not needed

1

e. Select the correct option for line 12 of the `printLevelOrder` method above. [1 mark]

i. `Node->left`

ii. `node->right`

iii. `node`

iv. `root`

1

v. None of above

f. Select the correct option for line 13 of the `printLevelOrder` method above. [1 mark]

i. `Node->left = q.front( )`

ii. `node->right != NULL`

iii. `node->left != NULL`

iv. `node != NULL`

v. `root == NULL`

vi. Not needed

1

g. Select the correct option for line 14 of the `printLevelOrder` method above. [1 mark]

i. `Node->left`

ii. `node->right`

iii. `node`

iv. `root`

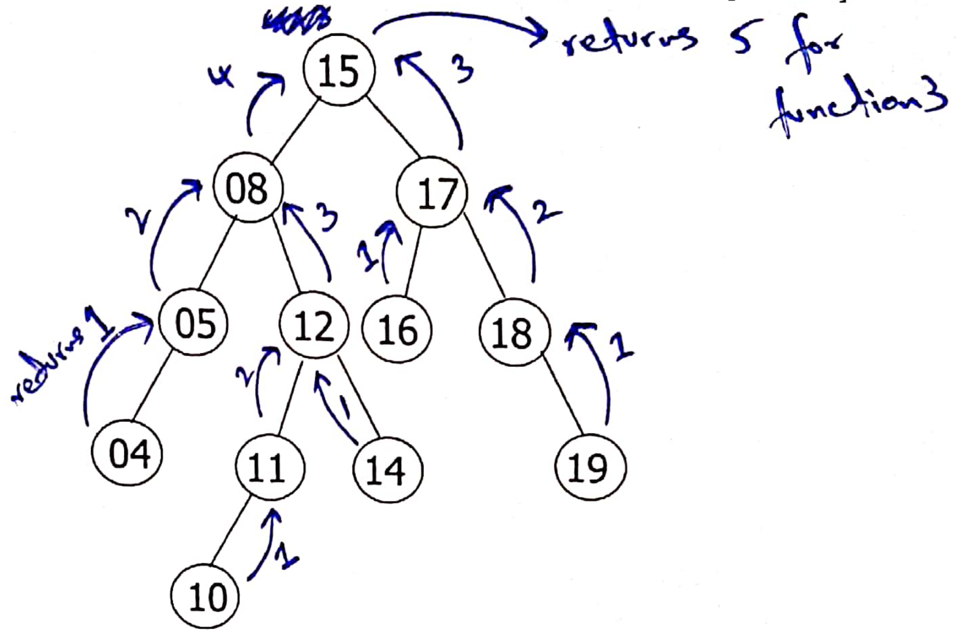
v. None of above

Rough Work (No marks will be awarded without properly understandable rough work):

2. Consider the following code snippet and answer the following questions. [5 marks]

```
void Function1 (node* ptr)
{
    int num = Function3(ptr);
    for (int i = 1; i <= num; i++)
        Function2(ptr, i);
}
void Function2 (node* ntr, int var)
{
    if (ntr == NULL)
        return;
    if (var == 1)
        cout << ntr->data << " , ";
    else if (var > 1) {
        Function2(ntr->left, var - 1);
        Function2(ntr->right, var - 1);
    }
}
int Function3(node* node)
{
    if (node == NULL)
        return 0;
    else {
        int lvar = Function3 (node->left);
        int rvar = Function3 (node->right);

        /* use the larger one */
        if (lvar > rvar) {
            return (lvar + 1);
        }
        else {
            return (rvar + 1);
        }
    }
}
int main()
{
    //create tree
    Function1(root);
    return 0;
}
```

a. Given the following tree, what will be the output of the above code snippet. No marks will be awarded without proper dry run indicating the status of the system stack. [3 marks]



i. 15 , 08 , 05 , 04 , 12 , 11 , 10 , 14 , 17 , 16 , 18 , 19

ii. 04 , 05 , 08 , 10 , 11 , 12 , 14 , 15 , 16 , 17 , 18 , 19

iii. 15 , 08 , 17 , 05 , 12 , 16 , 18 , 04 , 11 , 14 , 19 , 10

iv. 04 , 05 , 10 , 11 , 14 , 12 , 08 , 16 , 19 , 18 , 17 , 15

v. 15 , 17 , 08 , 18 , 16 , 12 , 05 , 19 , 14 , 11 , 04 , 10

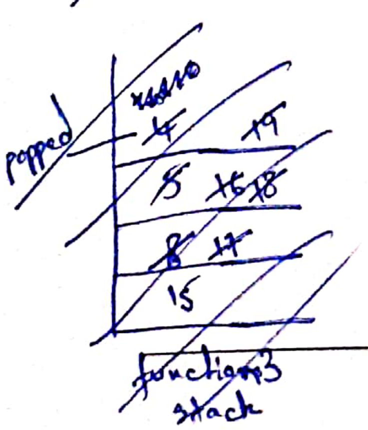
vi. None of above

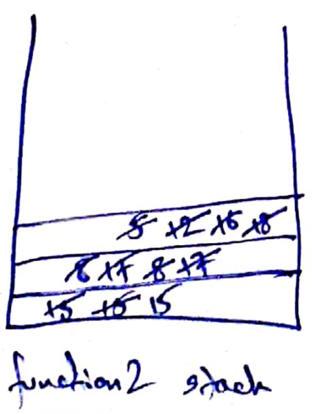
3

Dry run:

lvar = 0 + 1 + 1 + 1 + 0 + 0 + 0 + 0

rvar = 0 + 1 + 0 + 0 + 1 + 1 + 1 + 1



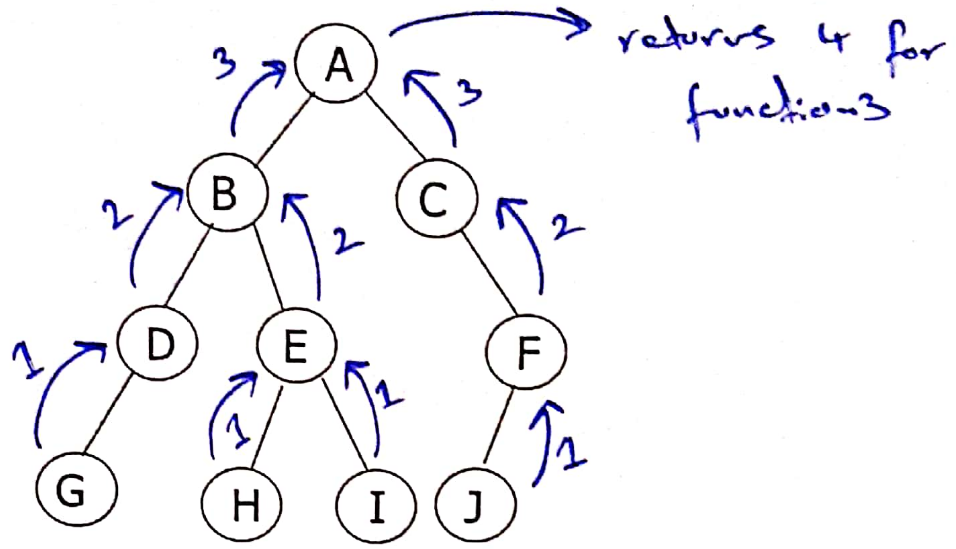


level order printing pattern

output:

15, 8, 17, 5, 12, 16, 18, 4, 11, 14, 19, 10

b. Given the following tree, what will be the output of the above code snippet. No marks will be awarded without proper dry run indicating the status of the system stack. [2 marks]



i. A , B , D , G , E , H , I , C , F , J

ii. G , D , H , I , E , B , J , F , C , A

iii. G , D , B , H , E , I , A , J , F , C

iv. A , B , C , D , E , F , G , H , I , J

v. A , C , B , F , E , D , J , I, H , G

vi. None of above

2

Dry run:

level order printing
as deduced from part (a)
thus output: A, B, C, D, E, F, G, H, I, J

3. Suppose we have numbers between 1 and 1000 in a binary search tree and want to search for the number 363. Which of the following sequence could not be the sequence of the node examined? No marks will be awarded without the proper justification. [1 mark]

a. 2, 252, 401, 398, 330, 344, 397, 363

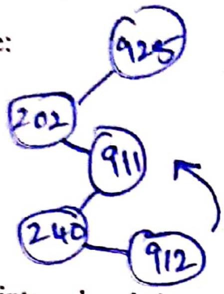
b. 924, 220, 911, 244, 898, 258, 362, 363

c. 925, 202, 911, 240, 912, 245, 258, 363

node 912 cannot come after 240 as 912 is larger than 911 which is ~~2~~ parent node. 240's larger

d. 2, 399, 387, 219, 266, 382, 381, 278, 36

Justification by drawing the tree:



cannot be greater than 911

4. In full binary search tree every internal node has exactly two children. If there are 100 leaf nodes in the tree, how many internal nodes are there in the tree? No marks will be awarded without the proper justification. [1 mark]

a. 25

b. 49

c. 99

d. 101

Justification:

formula for internal nodes is ~~#~~ leaf nodes - 1

5. The height of a BST is given as h. The maximum no. of nodes possible in the tree is? No marks will be awarded without the proper justification. [1 mark]

a. 2^{h-1} -1

b. 2^{h+1} -1

c. h + 1

d. h -1

e. 2^h + 1

f. 2^{h-1} +1

g. 2^h+1

Justification by drawing the tree:

formula for max nodes is $2^{height+1} - 1$

6. The height of a BST is given as h. The minimum no. of nodes possible in the tree is? No marks will be awarded without the proper justification. [1 mark]

a. 2^{h-1} -1

b. 2^{h+1} -1

c. h + 1

d. h -1

e. 2^h + 1

f. 2^{h-1} +1

g. 2^h+1

1

Justification by drawing the tree:

formula for min nodes is h+1

7. In a full binary tree, every internal node has exactly two children. A full binary tree with 2n+1 nodes contains? No marks will be awarded without the proper justification. [1 mark]

a. n leaf node

b. n internal nodes

c. n-1 leaf nodes

d. n-1 internal nodes

Justification:

~~[illegible]~~ ~~[illegible]~~ 2n+1 gives ~~[illegible]~~ no. of nodes where n = height so

8. The run time for traversing all the nodes of a binary search tree with n nodes and printing them in an order is: No marks will be awarded without the proper justification. [1 mark]

a. O(n log(n))

b. O(n)

c. O(√n)

d. O(log(n))

~~[illegible]~~ as traversal is recursive, it has logn complexity

Justification:

recursive traversal

9. If n elements are sorted in a binary search tree. What would be the asymptotic complexity to search a key in the tree? No marks will be awarded without the proper justification. [1 mark]

a. O(1)

b. O(log n)

c. O(n)

d. O(nlogn)

Justification:

as bst is ordered (left smaller, right larger) thus searching has n complexity.

10. If n elements are sorted in a balanced binary search tree. What would be the asymptotic complexity to search a key in the tree? No marks will be awarded without the proper justification. [1 mark]

a. O(1)

b. O(logn)

c. O(n)

d. O(nlogn)

Justification

bst ordered hence searching not ~~a~~ very intensive.

National University of Computer and Emerging Sciences

FAST School of Computing   Fall-2022   Islamabad Campus

Question 2 [20 Marks]

a. Consider the following code of an array-based stack class and main to answer the following questions.

```
class Stack
 {
private:
char *stackArray;
int stackSize;
int top;

public:
Stack(int size)
{
stackArray = new char[size];
stackSize = size;
top = -1;
}

void push(char val)
{
if (isFull())
cout << "The stack is full.\n";
else
{
top++;
stackArray[top] = val;
}
}

void pop(char &val)
{
if (isEmpty())
 cout << "The stack is empty.\n";
else
{
val = stackArray[top];
top--;
}
}

bool isFull()
{
return (top == stackSize - 1);

}

bool isEmpty()
{
return (top == -1);
}
```

```
void makeNull()
{
top = -1;
}
};


void main()
{
double num;
double num2;
char ch;
Stack stack(10);

ifstream infile;
infile.open("data.txt");
if (!infile)
{
cout << "The input file does not exist." << endl;
return;
}
//reads file line by line and puts the first value in num and
//second in ch
infile >> num >> ch;

num2 = num;
//loop till number of lines in file
while (infile) //Loop 1
{
if (num > num2)
{
stack.makeNull();
if (!stack.isFull()) {
stack.push(ch); }
num2 = num;
}
else if (num == num2) {
if (!stack.isFull()){
stack.push(ch);
}
else
{
cout << "Stack overflows "<< endl;
return;
}
}
//reads next file line and puts the first
//value in num and second in ch
infile >> num >> ch;
}

cout << num2 << endl;
```

```
while (!stack.isEmpty()) //Loop 2
{
char v;
stack.pop(v);
cout << v << "  "<< num2 << endl;
}
cout << endl;
return;
}
```

data.txt input file is shown below :

```
3.4 R
3.2 K
2.5 C
3.4 T
3.8 J
3.8 M
3.6 P
3.5 D
3.8 B
```

i. For the given data.txt file, what will be the values in the stack and what will be the values of num, num2 and ch after each iteration of Loop 1 of main(). [9 marks]

Iteration number	num	num2	ch
After Iteration 1			
After Iteration 2			
After Iteration 3			
After Iteration 4			
After Iteration 5			
After Iteration 6			
After Iteration 7			
After Iteration 8			
After Iteration 9			

006

stack:

	After Iteration 1	After Iteration 2	After Iteration 3	After Iteration 4	After Iteration 5
0					
1					
2					
3					
4					
5					
6					
7					
8					
9					
Top					

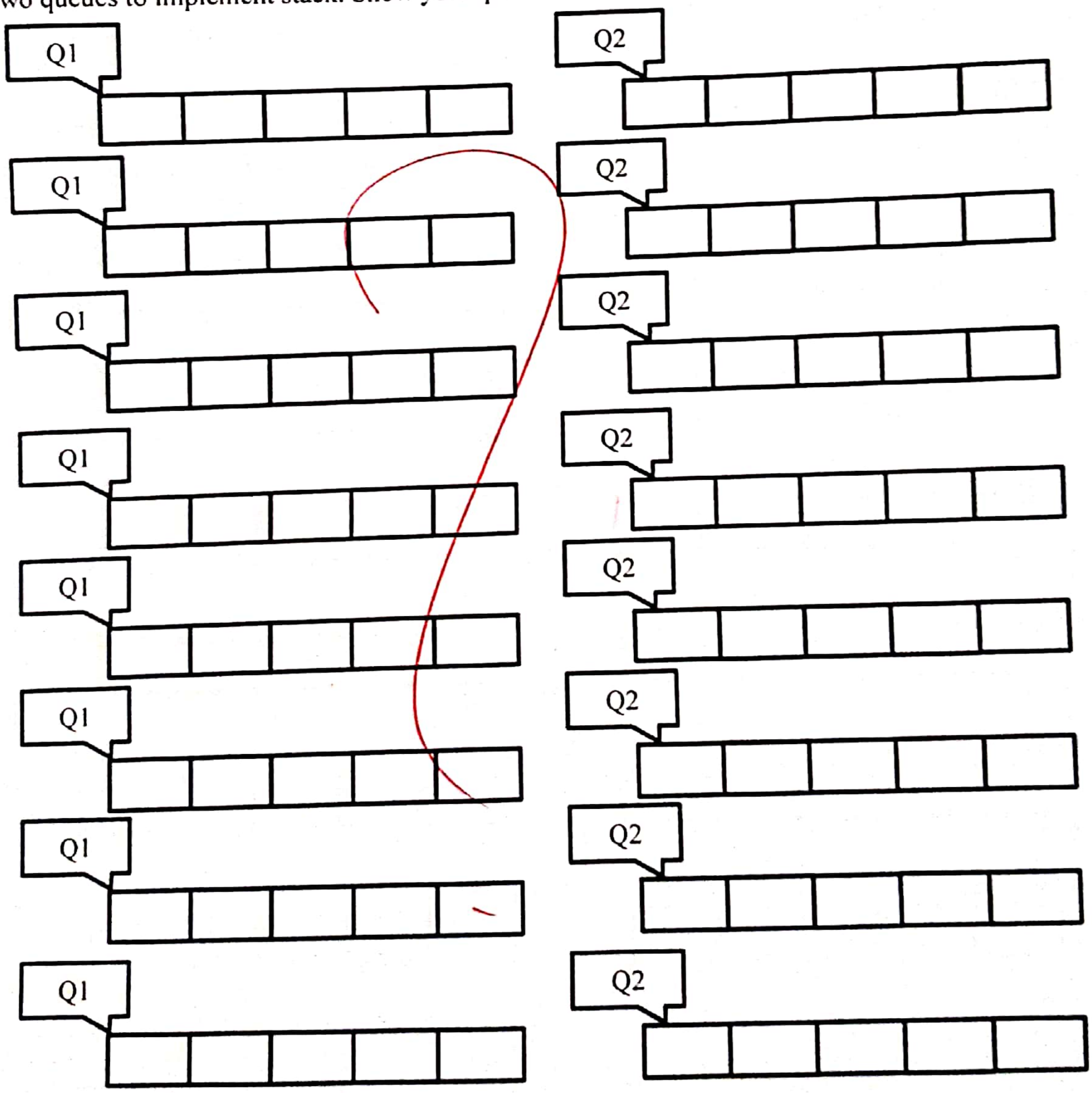
	After Iteration 6	After Iteration 7	After Iteration 8	After Iteration 9
0				
1				
2				
3				
4				
5				
6				
7				
8				
9				
Top				

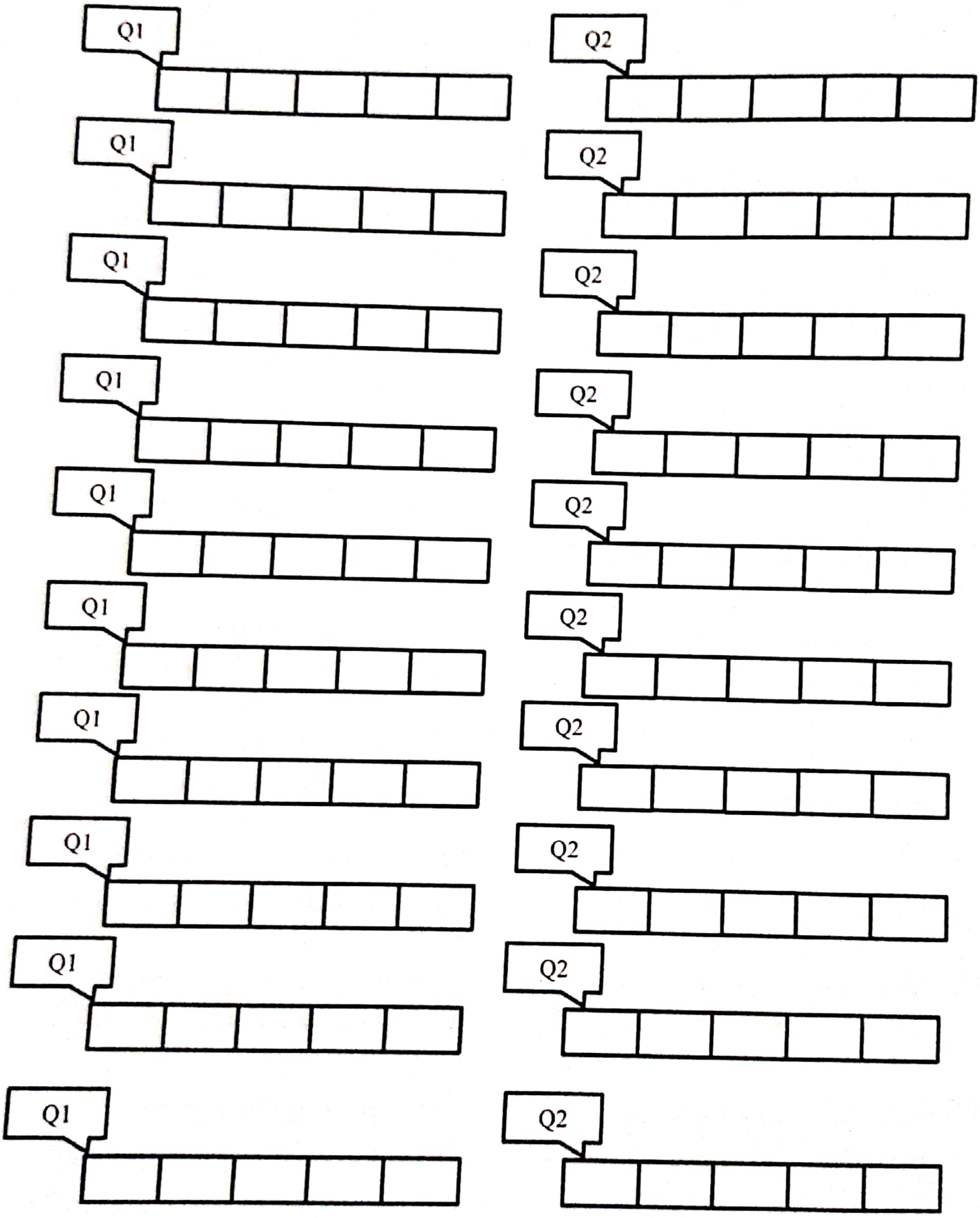
ii. What is the output of Loop 2 of main()? (3)

b. Following data must be processed through a stack in the given sequence

- Push(3)
- Push(32)
- Push(16)
- Push(5)
- Pop()
- Push(9)
- Pop()
- pop()

However, you only have two queues available (each of size=5) and no stack. Your task is to use these two queues to implement stack. Show your queues at each push and pop attempt. (8 marks)





NOTE: if you still need extra queues to show your working, you can extend your working on rough sheet at the end. Use same representations of Queue 1 and 2

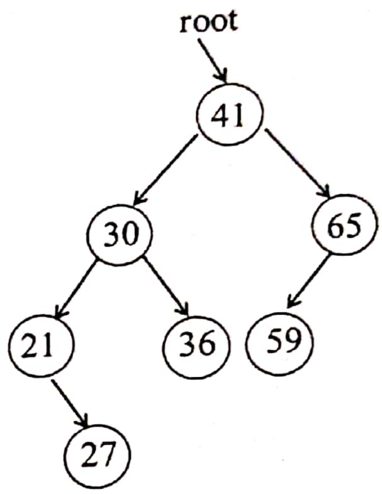
National University of Computer and Emerging Sciences

FAST School of Computing Fall-2022 Islamabad Campus

Question 3 [20 Marks]

Given a code of BST.

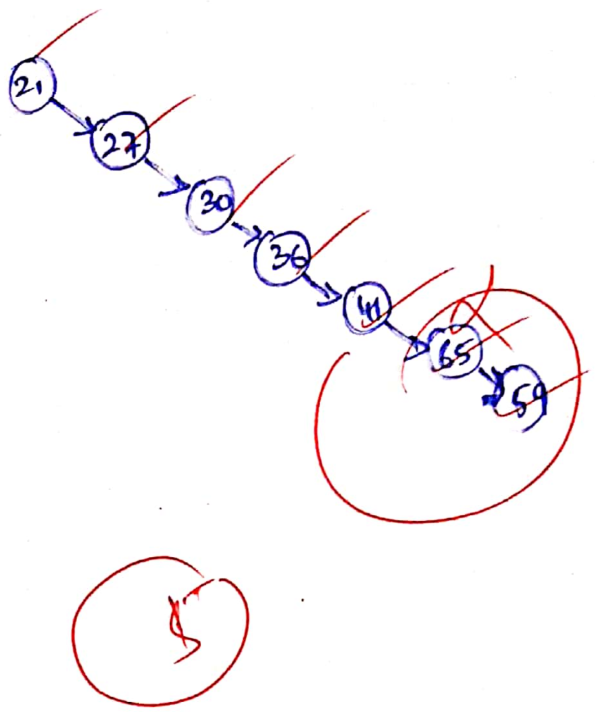
```
void UpdateTree() {
    if (root != NULL) {
        node* pre = NULL;
        root = UpdateTree(root, pre);
    }
}

node* UpdateTree(node* curr, node* pre) {
    if (curr->left == NULL && curr->right == NULL)
        return curr;

    node* r1 = NULL;
    node* r2 = NULL;
    if (curr->left != NULL)
        r1 = UpdateTree(curr->left, curr);
    if (curr->right != NULL)
        r2 = UpdateTree(curr->right, curr);
    if (r1 == NULL)
        return curr;
    if (r1->left != NULL)
    {
        r1->left->right = r1;
        r1 = r1->left->right;
        r1->left = NULL;
    }
    node* tmp = r1;
    while (tmp->right != NULL)
        tmp = tmp->right;
    tmp->right = curr;
    curr->right = r2;
    curr->left = NULL;
    return r1;
}
```



National University of Computer and Emerging Sciences

FAST School of Computing **Fall-2022** **Islamabad Campus**

1. Redraw the following BST after execution of the code given below. [14 Marks]



2. Is the tree still BST? [2 Mark]

No

3. If there are N number of nodes, what will be the worst time complexity to search an item in updated tree? [2 Mark]

N

4. What is the worst time complexity of the above code? Justify your answer. [2 Mark]

N, as it is now a linked list